

MobCal Automation Installation and Usage

Written by Joseph A. Haun

June 27, 2025

1. INTRODUCTION

This document aims to document the installation and usage of the MobCal Makefile automation. This automation provides a so-called "batch-mode" which sets standard parameters over which to analyze the CCS. This automation can also be utilized in a so-called "semi-manual" mode which allows the user to change standard parameters as desired.

2. TABLE OF CONTENTS

Contents

1	Introduction	1
2	Table of Contents	1
3	Installation	2
3.1	Operating System	2
3.2	Required Programs	2
4	Usage	3
4.1	Commands	3
4.2	Makefile Parameters	7
4.3	Examples	8

3. INSTALLATION

This section details the steps to create an Ubuntu machine which is capable of running the MobCal automation. Steps are provided from the best recollection. Other Linux distributions are likely possible, but development machine was an Ubuntu virtual machine.

3.1 Operating System

First the Ubuntu operating system must be downloaded from [Canonical's Website](#). The desktop version was used, but the server version could also be used. Follow the installation instructions provided by Canonical.

If a virtual machine is to be used, then [Oracle VirtualBox](#) must also be downloaded and installed. Virtual machines are handy if the user has a Windows machine and does not want to dual-boot. It also provides finer control over the RAM and Cores used in the system which will be automatically detected by the automation.

3.2 Required Programs

Once the operating system has been installed, the version control tool git will need to be installed. This can be done by opening a Terminal (CTL + ALT + T) and typing in the following command.

```
sudo apt-install git
```

Once installed, the MobCal repository can be cloned. This could also be manually downloaded from GitHub, but this may break the automatic updating of the repository. Once again, this is done in the Terminal window.

```
git clone git@github.com:HopkinsLaboratory/MobCal-MPI_21.git
```

Follow all steps listed under Sections 3.1 and 3.2 of the MobCal-MPI installation manual with the exception of GitHub desktop (Section 3.2.2) which is replaced by the command-line utility git installed above.

For the sdf2xyz2sdf installation, follow the instructions on the [SourceForge Linux binaries](#) section.

For openbabel, the Ubuntu package should suffice. Once again, this is done in the Terminal window. If not, then I can figure out how I actually got the package installed, but I think I just did this.

```
sudo apt install openbabel
```

For the Fortran compilation, follow the steps listed in Section 4.1.4 of the MobCal-MPI manual for OpenMPI and gfortran. The Intel fortran compiler was not used due to limited compatibility with development system. Automation commands assume gfortran, but could be changed as described in Section 4.2.

For the Orca installation, go to the [Orca Forum](#) and follow their installation instructions for a Linux system. Version used was 5.0.1 if I remember correctly. I have not tested any later versions, but they should all work so long as the syntax has not changed. Once downloaded, add the Orca files location to the system PATH variable. This can be done by adding the following line to the `/.bashrc` file.

```
export PATH="$PATH:<orca location>"
```

An example of this, for all utilities manually installed, is as follows for my development system.

```
export PATH="$PATH:/shared/opnmpi_64/bin"  
export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:/shared/opnmpi_64/lib"  
export PATH="$PATH:/home/ion-library/Documents/orca"  
export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:/home/ion-library/Documents/orca"  
export PATH="$PATH:/home/ion-library/Documents/open3dtools/bin"
```

This ensures that the *which* utility can find these executables. The automation makes use of this to allow the user freedom to install where desired/needed.

4. USAGE

This section first details the commands, see Section 4.1, then the user-controllable parameters, see Section 4.2, and then several usage examples, see Section 4.3.

4.1 Commands

Each command listed is implemented as a [Makefile recipe](#). In general, this means that every command listed, must be prefaced by using the command-line implementation of **make**.

It is also recommended that the user utilize the **-directory** and **-makefile** command line flags to dynamically redirect the Make utility to a separate directory. This helps keep the directory clean as many of the underlying utilities (i.e. crest, orca, etc.) will create temporary files for their calculations in the current directory. By passing in the **-directory** flag, the user tells the Make utility to call any other command in that directory. When changing this directory, the Make utility then looks in that new directory for the Makefile. This is where the **-makefile** flag comes into play as the user can provide the file path of the current directory. This sounds odd, but it is used heavily by the "batch-mode" to keep the calling directory clean. As a result of this, it is recommended that any file path be an [absolute file path](#).

It is worth noting that the exception to this recommendation of changing directory with the **-directory** flag are process-chemical, update, download-chemical, and convert-smiles recipes. These recipes place generated files within a build directory automatically or appropriately clean up temporary files.

Each command will check for the necessary variables. This check is done by checking that the variable is *not* a blank string. If the value is not present, the recipe will print an error message to the terminal and cease all operations. No other checking is done on the validity of the value and crashes should be expected if improper values are provided. Variables are *case sensitive* and should be entered as shown on the error message.

To account for spaces in file paths, it is recommended to encapsulate all inputs in "double-quotation marks" so that the command-line does not interpret these as separate values. Makefile recipes automatically do this internally, so only user-inputs need to be mindful of this behavior.

1. update

Purpose: Update the current repository by pulling the latest changes from the remote repository with the command-line git. Typically this will be GitHub, but is ultimately controlled by the git utility.

Required Inputs: None

2. process-chemical

Purpose: Process the provided input file in the "batch-mode" of operation. Protonation and deprotonation sites will be calculated and passed through the automation pipeline to generate CCS values for each unique file.

Required Inputs:

INPUT_FILE - An SDF or XYZ file which can be used by crest. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

3. generate-charge-particles

Purpose: Generate possible ionized versions of the molecule in an XYZ file format.

Required Inputs:

INPUT_FILE - An SDF or XYZ file which can be used by crest and describes the molecule to be ionized. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

OUTPUT_DIRECTORY - The location where output files will be saved.

CHARGE - A numerical value which indicates the charge of the ion.

4. generate-conjugate

Purpose: Generate a protonated or deprotonated version of the molecule in an XYZ file format.

Required Inputs:

INPUT_FILE - An SDF or XYZ file which can be used by crest and describes the molecule to be ionized. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

OUTPUT_DIRECTORY - The location where output files will be saved.

CHARGE - A numerical value which indicates the charge of the ion.

5. generate-ion

Purpose: Generate a cation or anion of the molecule in an XYZ file format.

Required Inputs:

INPUT_FILE - An SDF or XYZ file which can be used by crest and describes the molecule to be ionized. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

OUTPUT_DIRECTORY - The location where output files will be saved.

CHARGE - A numerical value which indicates the charge of the ion.

6. generate-dimer

Purpose: Generate a dimer with the provided molecule and neutral files in an XYZ file format.

Required Inputs:

INPUT_FILE - An XYZ file which can be used by crest which describes the ionized molecule. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

NEUTRAL_FILE - An XYZ file which can be used by crest which describes the non-ionized molecule. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

OUTPUT_DIRECTORY - The location where output files will be saved.

CHARGE - A numerical value which indicates the charge of the ion.

7. generate-conformers

Purpose: Generate possible conformers of the provided file which meet the similarity threshold percentage.

Required Inputs:

INPUT_FILE - An SDF or XYZ file which can be used by crest which describes the ionized molecule. Typically generated by recipes provided in the Makefile: convert-smiles and download-chemical.

OUTPUT_DIRECTORY - The location where output files will be saved.

CHARGE - A numerical value which indicates the charge of the ion.

SIMILARITY_THRESHOLD - A numerical value which is the percentage similarity allowed for conformers.

8. generate-orca-inputs

Purpose: Process the files created by generate-conformers recipe into the file formats used by the orca utility for calculations.

Required Inputs:

INPUT_DIRECTORY - The location where files are stored.

OUTPUT_DIRECTORY - The location where output files will be saved.

CHARGE - A numerical value which indicates the charge of the ion.

MULTIPLICITY - A numerical value which indicates the multiplicity of the molecules in the input directory.

9. generate-orca-outputs

Purpose: Process the files created by generate-orca-inputs recipe with the orca utility.

Required Inputs:

INPUT_DIRECTORY - The location where files are stored.

OUTPUT_DIRECTORY - The location where output files will be saved.

10. generate-mobcal-inputs

Purpose: Process the files created by generate-orca-outputs recipe with the MobCal scripts to create the MFJ inputs for MobCal calculations.

Required Inputs:

INPUT_DIRECTORY - The location where files are stored.

MFJ_CYCLES - The MobCal Cycles Parameter. See GUI Documentation.

CHARGE_SETTING - The MobCal Charge Setting parameter. See GUI Documentation.

MFJ_VELOCITY_INTEGRATION - The MobCal Velocity Integration parameter. See GUI Documentation.

MFJ_IMPACT_INTEGRATION - The MobCal Impact Integration parameter. See GUI Documentation.

MFJ_CARRIER_GAS - The MobCal Carrier Gas parameter. See GUI documentation.

MFJ_EMPIRICAL_CORRECTION - The MobCal Empirical Correction parameter. See GUI Documentation.

MFJ_TEMPERATURE - The MobCal Temperature parameter. See GUI Documentation.

11. generate-mobcal-outputs

Purpose: Process the MFJ files created by generate-mobcal-inputs recipe with the MobCal executable.

Required Inputs:

INPUT_DIRECTORY - The location where files are stored.

OUTPUT_DIRECTORY - The location where output files will be saved.

12. convert-smiles

Purpose: Convert a provided SMILES string to an SDF file suitable for automation pipeline.

Required Inputs:

CHEMICAL_STRING - The SMILES string to use for file generation.

CHEMICAL_NAME - The name of the chemical being generated.

13. add-chemical

Purpose: Helper recipe for the convert-smiles recipe to add generated SDF file to standard location.

Required Inputs:

INPUT_FILE - The chemical file to store.

CHEMICAL_NAME - The name of the chemical being generated.

14. download-chemical

Purpose: Download a chemical from the [PubChem database](#) through the PUG REST API.

Required Inputs:

CHEMICAL_CID - The CID number shown on PubChem's website.

15. process-pubchem-data

Purpose: Helper recipe for download-chemical to sort out metadata pulled from the PUG REST API in a README.md file.

Required Inputs:

PUBCHEM_DATA - The file which stores the PubChem metadata.

OUTPUT_DIRECTORY - The location where to store the README file.

16. check-chemical-exists

Purpose: Helper recipe for download-chemical and add-chemical recipes to prevent addition of chemicals already present in the standard location.

Required Inputs:

CHEMICAL_NAME - The chemical name.

17. check-input

Purpose: Helper recipe for download-chemical and add-chemical recipes to prevent addition of chemicals already present in the standard location.

Required Inputs:

INPUT_VALUE - The value to check.

INPUT_VARIABLE - The name of the checked.

18. charge-to-conjugate

Purpose: Helper recipe to convert a charge string to a conjugate string.

Required Inputs:

CHARGE - The charge string to convert.

19. charge-to-ion

Purpose: Helper recipe to convert a charge string to a radical ion string.

Required Inputs:

CHARGE - The charge string to convert.

20. remove-build-directory

Purpose: Helper recipe for process-chemical that removes the individual build directories to preserve harddrive space.

Required Inputs:

DIRECTORY - The directory to remove.

21. divider

Purpose: Helper recipe to print a divider line between commands for readability during automation.

Required Inputs: None

22. clean

Purpose: Removes automation standard build directory.

Required Inputs: None

4.2 Makefile Parameters

This section details Makefile variables which *may* be changed by the user, but will be populated by the Makefile itself normally. Largely, they control the basic locations of scripts and utilities necessary for the automation to function. This list does not include Makefile variables which are required by the recipes listed in Section 4.1.

SCRIPTS_DIRECTORY - The location for all scripts used in the automation.

BUILD_DIRECTORY - The location for standard automation pipeline to use for operations to keep directory clean.

FORTRAN_DIRECTORY - The location of the MobCal Fortran source code.

RESULTS_DIRECTORY - The location for all results to be placed by the automation.

CHEMICAL_STRUCTURE_DIRECTORY - The location for the chemical structure files to be stored by default for download-chemical and convert-smiles recipes.

MPIRUN - The MPIRUN executable/command.

FORTRAN_COMPILER - The FORTRAN compiler to use for MobCal executable.

FORTRAN_FLAGS - Flags to pass to the FORTRAN compiler during MobCal executable compilation.

OPEN_BABEL - The openbabel utility path. Determined by examining system PATH variable.

ORCA - The orca utility path. Determined by examining system PATH variable.

SDF2TINKERXYZ - The sdf2tinkerxyz utility path. Determined by examining system PATH variable.

EMPIRICAL_CORRECTIONS - The location of the script used to determine if empirical corrections should be applied.

ION_SPLITTER - The location of the script used to split ion files when generating conformers.

COMPARE_MASSES - The location of the script used to compare masses when generating conformers.

GFJ_ORCA_CONVERSION - The location of the script used to convert GJF files into Orca input files.

THERMO_CHEM_EXTRACTOR - The location of the script used to extract thermochem information from Orca output files.

MFJ_CREATOR - The MFJ creator script used.

MOBCAL_OUTPUT_ANALYZER - The MobCal output analyzer script usage.

DIMER_APPEND - The script used to generate dimer files.

CORE_COUNTER - The script used to determine the available processor cores.

MEM_COUNTER - The script used to determine the available memory per parallel process.

XYZ_FILE_FORMAT_CHECKER - The script used to determine if the file is an XYZ formatted file.

MFJ_CREATOR - The location of the script used to generate MFJ files from Orca output files.

DIMER_APPEND - The location of the script used to concatenate XYZ files for dimer calculations.

CORE_COUNTER - The location of the script used to determine available cores.

MEM_COUNTER - The location of the script used to determine available RAM.

AVAILABLE_CORES - The number of cores available for parallel computations.

AVAILABLE_RAM - The megabytes of available RAM available for parallel computations.

DIVIDER_COUNT - The number of dashes to print for the divider.

CONFORMERS_FILE - The default crest conformers file.

ION_SPLITTER_OUTPUT_DIRECTORY - The directory created during ion splitting.

MASS_COMPARISON_OUTPUT_DIRECTORY - The directory created during mass comparisons.

MFJ_CREATOR_OUTPUTS_DIRECTORY - The directory created during MFJ file creation.

FORTTRAN_FLAGS - Flags provided to the Fortran compiler for the compilation of the MobCal executable.

MOBCAL_INPUT_FILE - The MobCal file created to serialize runtime.

DIMER_FILE - The temporary dimer file created.

MFJ_CREATOR_LOG - The MFJ creator logfile name.

PIPELINE_SIMILARITY_THRESHOLD - The similarity threshold used during process-chemical operations. Defaults to 95.

PIPELINE_MFJ_CHARGE_SETTING - The MFJ charge setting used during process-chemical operations. Defaults to 'calc' per MobCal-MPI documentation.

PIPELINE_MFJ_CYCLES - The MFJ cycles setting used during process-chemical operations. Defaults to 10.

PIPELINE_MFJ_VELOCITY_INTEGRATION - The MFJ velocity integration setting used during process-chemical operations. Defaults to 200.

PIPELINE_MFJ_IMPACT_INTEGRATION - The MFJ impact integration setting used during process-chemical operations. Defaults to 1024.

PIPELINE_MFJ_CARRIER_GAS - The MFJ carrier gas setting used during process-chemical operations. Defaults to 2.

PIPELINE_MFJ_TEMPERATURE - The MFJ temperature string used during process-chemical operations. Defaults to "298 1008 71" to a 5 C° temperature distribution.

4.3 Examples

This section provides some examples of how to use the Makefile recipes as well as the basic syntax necessary for changing Makefile variables which are either required or optional.

Example: make update

Example: make download-chemical CHEMICAL_CID=4133

Example: make convert-smiles CHEMICAL_STRING="COC(=O)C1=CC=CC=C1O"
CHEMICAL_NAME="methyl salicylate"

Example: make process-chemical INPUT_FILE="<path to input file>"